Ex.No: 3.1

Caesar Cipher

AIM:

To implement a Java program for encrypting and decrypting a text message using Caesar Cipher for security purposes.

ALGORITHM:

Encryption:

1. Read the plaintext string.
2. Read the key (shift value).
3. For each character in the string:
   - If it is an uppercase letter, shift within A–Z.
   - If it is a lowercase letter, shift within a–z.
   - Leave other characters unchanged.
4. Display the encrypted text.

Decryption:

1. Read the encrypted text.
2. Use the same key.
3. Shift characters backward by the key.
4. Display the decrypted text.

CODE:

```java
import java.util.*;

class Caesar {

    static String encrypt(String text, int k) {
        String result = "";

        for (char c : text.toCharArray()) {
            if (Character.isLetter(c)) {
                char base = Character.isUpperCase(c) ? 'A' : 'a';
                result += (char)(base + (c - base + k) % 26);
            } else {
                result += c;
            }
        }
        return result;
    }

    static String decrypt(String text, int k) {
        return encrypt(text, 26 - k);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter text: ");
        String t = sc.nextLine();

        System.out.print("Enter shift: ");
        int k = sc.nextInt();
```

```
        String enc = encrypt(t, k);
        String dec = decrypt(enc, k);

        System.out.println("Encrypted: " + enc);
        System.out.println("Decrypted: " + dec);
    }
}
```

OUTPUT:

```
Enter the text:
hello
Enter the shift value:
3

Encrypted text: khoor
Decrypted text: hello
```

Ex.No: 3.2

Hill Cipher

AIM:

To implement a Java program for encrypting and decrypting a text message using Hill Cipher.

ALGORITHM:

Encryption:

1. Read the plaintext and key matrix.
2. Convert plaintext to uppercase and remove non-alphabetic characters.
3. Pad the text if required.
4. Convert characters to numbers (A = 0 to Z = 25).
5. Multiply plaintext blocks with the key matrix and take modulo 26.
6. Convert result back to characters.

Decryption:

1. Read ciphertext and key matrix.
2. Find inverse of key matrix modulo 26.
3. Multiply ciphertext with inverse matrix.
4. Convert result back to plaintext.

CODE:

```
import java.util.*;

class Hill {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter text (even length, uppercase): ");
        String t = sc.nextLine().toUpperCase();

        int[][] k = {{3,3},{2,5}};  // key matrix
        int[][] inv = {{15,17},{20,9}}; // inverse matrix (for decryption)

        String enc = "", dec = "";
```

```java
        // ENCRYPT
        for(int i=0;i<t.length();i+=2){
            int a = t.charAt(i)-'A';
            int b = t.charAt(i+1)-'A';

            int x = (k[0][0]*a + k[0][1]*b)%26;
            int y = (k[1][0]*a + k[1][1]*b)%26;

            enc += (char)(x+'A');
            enc += (char)(y+'A');
        }

        // DECRYPT
        for(int i=0;i<enc.length();i+=2){
            int a = enc.charAt(i)-'A';
            int b = enc.charAt(i+1)-'A';

            int x = (inv[0][0]*a + inv[0][1]*b)%26;
            int y = (inv[1][0]*a + inv[1][1]*b)%26;

            dec += (char)(x+'A');
            dec += (char)(y+'A');
        }

        System.out.println("Encrypted: " + enc);
        System.out.println("Decrypted: " + dec);
    }
}
```

OUTPUT:

Plaintext: HELP
Encrypted: HIAT
Decrypted: HELP

Ex.No: 3.3

Substitution Cipher

AIM:

To implement a Java program for encrypting and decrypting text using substitution cipher.

ALGORITHM:

1. Read plaintext.
2. Replace each character using substitution key.
3. Display encrypted text.
4. Reverse process to decrypt.

CODE:

```java
import java.util.*;

class Sub {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        String t = sc.nextLine().toLowerCase();

        String a = "abcdefghijklmnopqrstuvwxyz";
```

```
        String b = "zyxwvutsrqponmlkjihgfedcba";

        String e = "", d = "";

        // ENCRYPT
        for(char c : t.toCharArray()){
            int i = a.indexOf(c);
            e += (i != -1) ? b.charAt(i) : c;
        }

        // DECRYPT
        for(char c : e.toCharArray()){
            int i = b.indexOf(c);
            d += (i != -1) ? a.charAt(i) : c;
        }

        System.out.println("Enc: " + e);
        System.out.println("Dec: " + d);
    }
}
```

OUTPUT:

Enter any string:
hello

Encrypted Text: svool
Decrypted Text: hello

Ex. No: 4.1

CONFIGURE UFW FIREWALL TO ALLOW ONLY WEB TRAFFIC (PORT 80 & 443)

Aim:

To configure a basic firewall using UFW (Uncomplicated Firewall) to allow only HTTP (port 80) and HTTPS (port 443) traffic and deny all other incoming connections.

Code:

```
import java.io.IOException;

class UFWFirewall {

    public static void main(String[] args) {
        try {
            System.out.println("Configuring UFW Firewall...");

            new ProcessBuilder("sudo", "ufw", "default", "deny", "incoming").start();
            new ProcessBuilder("sudo", "ufw", "default", "allow", "outgoing").start();
            new ProcessBuilder("sudo", "ufw", "allow", "80/tcp").start();
            new ProcessBuilder("sudo", "ufw", "allow", "443/tcp").start();
            new ProcessBuilder("sudo", "ufw", "enable").start();

            System.out.println("UFW configured to allow only web traffic.");

        } catch (IOException e) {
            System.out.println("Error configuring firewall");
        }
    }
}
```

Result:

UFW was successfully configured to allow only web traffic on ports **80 (HTTP)** and **443 (HTTPS)**, while blocking all other incoming connections.

Ex. No: 4.2

IDENTIFY PUBLICLY ACCESSIBLE WEBCAMS USING SHODAN.IO

Aim:

To identify insecure or publicly accessible webcam streams using Shodan, a search engine for Internet-connected devices.

Procedure:

Step 1: Visit Shodan Website

Go to: https://www.shodan.io

Step 2: Create Account

- Sign up or log in (free account)
- Required to access full search features

Step 3: Use Webcam Search Filters

Enter queries in the search bar:

```
webcamxp
```

```
netcam
```

```
port:554 has_screenshot:true
```

Other Useful Queries:

```
"webcam 7" country:"IN"
```

```
"IP Camera [Motion JPEG]" port:"8081"
```

Step 4: Analyze Results

Click on results (IP addresses) to view:

- Screenshot preview (if available)
- Location
- Port number
- Device manufacturer

Result:

Publicly accessible webcam streams were identified using Shodan search filters.
This highlights the importance of securing IoT devices and restricting unauthorized access.

Ex No: 5.1

ENCRYPT AND DECRYPT MESSAGE USING PGP KEYS

Aim:

To encrypt and decrypt messages or files using PGP keys and GPG commands in Linux, ensuring secure data confidentiality and integrity through public key cryptography.

Procedure:

Step 1: Check GPG Version

```
gpg --version
```

Step 1(a): Install GPG (if not installed)

```
sudo apt install gnupg
```

Step 2: Generate Key Pair

```
gpg --gen-key
```

Step 2(a): Construct User ID

Follow prompts:

- Real Name: Test user
- Email: test@example.com
- Press **O** to confirm

Step 2(b): Set Passphrase

- Enter passphrasem
- Confirm passphrase

Step 3: Verify Keys

```
gpg --list-keys
```

Step 4: Create Message File

```
nano message.txt
```

Step 4(a): Add Content

- Write your message
- Press **Ctrl + X**
- Press **Y**
- Press **Enter** to save

Step 5: Encrypt the File

```
gpg -e -r "Test User" message.txt
```

✅ This creates:
message.txt.gpg

Step 5(a): Verify File Creation

```
ls -l
cat message.txt.gpg
```

- Output will be **binary encrypted data** (not readable)

Step 6: Decrypt the File

```
gpg -d message.txt.gpg
```

Step 6(a): Enter Passphrase

- Enter your passphrase when prompted

Result:

The encryption and decryption of messages using PGP keys was successfully completed.

EX: 5.2

SECURE SSH ACCESS WITH FAIL2BAN AND KEY-BASED LOGIN

AIM:

To configure a Linux server to allow only key-based SSH authentication and secure it using Fail2Ban to prevent brute-force attacks.

PROCEDURE:

Step 1: Update the System

```
sudo apt update && sudo apt upgrade -y
```

Ensures all packages are updated for stability and security.

Step 2: Install OpenSSH Server

```
sudo apt install openssh-server -y
```

Installs SSH server for remote login.

Start and enable SSH:

```
sudo systemctl start ssh
sudo systemctl enable ssh
```

Step 3: Generate SSH Key Pair (Client Machine)

```
ssh-keygen -t rsa -b 4096
```

- Press Enter for default location (~/.ssh/id_rsa)
- Enter passphrase or leave empty

Step 4: Copy Public Key to Server

```
ssh-copy-id username@server_ip
```

Replace:

- username → your username

- server_ip → server IP

Find server IP:

```
ip a
```

Step 5: Disable Password Login

```
sudo nano /etc/ssh/sshd_config
```

Update/add:

```
PasswordAuthentication no
PermitRootLogin no
PubkeyAuthentication yes
```

Save and exit:

- Ctrl + O → Enter
- Ctrl + X

Restart SSH:

```
sudo systemctl restart ssh
```

Step 6: Test Key-Based Login

```
ssh username@server_ip
```

✅ Login should work without password

Step 7: Install Fail2Ban

```
sudo apt install fail2ban -y
```

Step 8: Configure Fail2Ban

```
sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local
sudo nano /etc/fail2ban/jail.local
```

Replace with:

```
[DEFAULT]
bantime = 600
findtime = 600
maxretry = 3
backend = systemd

[sshd]
enabled = true
port = ssh
logpath = %(sshd_log)s
```

Save and exit.

Step 9: Start and Enable Fail2Ban

```
sudo systemctl start fail2ban
sudo systemctl enable fail2ban
```

Step 10: Check Fail2Ban Status

```
sudo systemctl status fail2ban
sudo fail2ban-client status
```

Check SSH-specific status:

```
sudo fail2ban-client status sshd
```

RESULT:

SSH was successfully secured using key-based authentication and Fail2Ban, allowing only authorized access while preventing brute-force attacks.

Ex No: 6.1

Caesar Cipher using Shell Script

Aim:

To implement Caesar Cipher using a Linux shell script, perform encryption and decryption on a file, and verify correctness.

Procedure:

Step 1: Create Script File (No `vi`, chill)

```
nano caesar.sh
```

Simple Code (

```
#!/bin/bash

mode=$1
shift=$2
input=$3
output=$4

a=abcdefghijklmnopqrstuvwxyz
A=ABCDEFGHIJKLMNOPQRSTUVWXYZ

b=${a:$shift}${a:0:$shift}
B=${A:$shift}${A:0:$shift}

if [ $mode = "enc" ]; then
    echo "Encrypting..."
    tr "$a$A" "$b$B" < $input > $output
else
```

```
    echo "Decrypting..."
    tr "$b$B" "$a$A" < $input > $output
fi
cat $output
```

👉 This is way shorter + easier to remember:

- One `tr` instead of two
- No unnecessary echoes

Step 2: Make Script Executable

```
chmod +x caesar.sh
```

Step 3: Create Sample File

```
echo "Hello World" > plain.txt
```

Step 4: Encrypt

```
./caesar.sh enc 3 plain.txt cipher.txt
```

Output:

```
Khoor Zruog
```

Step 5: Decrypt

```
./caesar.sh dec 3 cipher.txt recover.txt
```

Output:

```
Hello World
```

Step 6: Verify

```
diff plain.txt recover.txt
```

👉 No output = Correct ✅

Result:

The Caesar Cipher was successfully implemented using a shell script, and encryption and decryption were verified.

Ex No: 6.2

Vigenère Cipher using Shell Script

Aim:

To implement Vigenère Cipher using a Linux shell script, perform encryption and decryption, and verify correctness.

Algorithm:

- Convert letters → numbers (A=0 to Z=25)
- Use keyword for shifting

Encryption:

```
C = (P + K) mod 26
```

Decryption:

```
P = (C - K + 26) mod 26
```

- Non-alphabet characters remain unchanged

Procedure:

Step 1: Create Script File

```
nano vigenere.sh
```

Simple Code

```
#!/bin/bash

mode=$1
key=$(echo $2 | tr a-z A-Z)
input=$3
output=$4

alpha=ABCDEFGHIJKLMNOPQRSTUVWXYZ

line=$(cat $input)
j=0

for ((i=0;i<${#line};i++)); do
  c=${line:$i:1}

  if [[ $c =~ [A-Za-z] ]]; then
    p=$(expr index "$alpha" "${c^^}")
    k=$(expr index "$alpha" "${key:$((j%${#key})):1}")

    if [ "$mode" = "enc" ]; then
      v=$(( (p+k-2)%26 ))
    else
      v=$(( (p-k+26)%26 ))
    fi

    echo -n "${alpha:$v:1}"
    ((j++))
  else
    echo -n "$c"
  fi
done > $output

echo
cat $output
```

Step 2: Make Executable

```
chmod +x vigenere.sh
```

Step 3: Create Sample File

```
echo "hello world" > plain.txt
```

Step 4: Encrypt

```
./vigenere.sh enc KEY plain.txt cipher.txt
```

Step 5: Decrypt

```
./vigenere.sh dec KEY cipher.txt out.txt
```

Step 6: Verify

```
diff plain.txt out.txt
```

👉 No output = Correct ✅

Result:

The Vigenère Cipher was successfully implemented using a shell script, and encryption and decryption were verified.

Ex. No: 7.1

Monitoring HTTP/HTTPS Messages Using Wireshark

Aim:

To capture and analyze HTTP/HTTPS messages exchanged between a browser and the website https://www.sathyabama.ac.in using Wireshark.

Procedure & Observation:

Step 1: Open Wireshark

- Launch Wireshark on your system

Step 2: Select Network Interface

- Choose the active network interface (connected to internet)

Step 3: Start Packet Capture

- Click **Start Capture** on the selected interface

Step 4: Visit Website

- Open browser
- Go to: https://www.sathyabama.ac.in
- Wait until the page fully loads

Step 5: Stop Capture

- Return to Wireshark
- Stop packet capture

Step 6: Apply Filters

Filter HTTPS/TLS traffic:

```
tls
```

Filter only handshake messages:

```
tls.handshake
```

Step 7: Analyze Packets

- **Packet ~1217** → TLSv1.2 Application Data begins
- **Packets 1218–1223** → Encrypted Application Data
- **Packet 1219** → TCP ACK (confirms delivery)

Observation:

- TLS handshake is visible initially
- After handshake → all communication is encrypted
- No readable HTTP content (due to HTTPS security)

Result:

The HTTPS communication with **www.sathyabama.ac.in** was successfully captured using Wireshark. TLS handshake and encrypted packets were observed, confirming that HTTPS ensures secure communication.

Ex. No: 7.2

Determine Highest Traffic Port While Browsing YouTube Using Wireshark

---

Aim:

To capture and analyze network traffic using Wireshark and determine the port number that carries the highest amount of traffic during YouTube browsing.

Requirements:

- Wireshark installed
- Active internet connection
- Access to YouTube

Algorithm / Procedure:

Step 1: Start Wireshark

- Open Wireshark
- Select the active network interface (Wi-Fi/Ethernet)

Step 2: Start Capture

- Click **Start Capturing**

Step 3: Generate Traffic

- Open browser
- Go to YouTube
- Play videos for a few minutes

Step 4: Stop Capture

- Return to Wireshark
- Stop packet capture

Step 5: Apply Filters (Optional)

For HTTPS traffic:

```
tcp.port == 443
```

For QUIC traffic:

```
udp.port == 443
```

Step 6: Analyze Traffic

- Go to **Statistics** → **Endpoints**
- Sort by **Bytes column**

- Identify which port has the highest traffic

Observation:

- Majority of packets are sent over **Port 443**
- Port 443 handles both:
  - HTTPS (TCP)
  - QUIC (UDP)

Output:

The highest traffic while browsing YouTube occurred on **Port 443**, which is used for secure communication (HTTPS and QUIC).

Result:

Thus, the port carrying the highest traffic while browsing YouTube using Wireshark was identified as **Port 443**. The experiment was successfully executed.

Ex.No: 8.1

Image LSB Steganography

Aim:

Hide a secret message inside an image using LSB steganography (Steghide) and extract and verify the hidden message.

STEP 1: Install Steghide in Ubuntu

```
sudo apt update
sudo apt install steghide
```

Verify installation:

```
steghide --version
```

STEP 2: Prepare Files

Choose a Cover Image

Use an image like:

```
cover.jpg
```

(JPEG or BMP recommended)

Create a Secret Message File

```
echo "This is a hidden message inside the image." > secret.txt
```

Check contents:

```
cat secret.txt
```

STEP 3: Hide the Secret Message

steghide embed -cf cover.jpg -ef secret.txt -sf stego.jpg

- `-cf` → cover file
- `-ef` → file to embed
- `-sf` → output stego file

You will be prompted to:

- Enter a passphrase
- Confirm the passphrase

✅ The secret data is now embedded inside the image.

STEP 4: Verify Embedded Data

```
steghide info stego.jpg
```

This shows:

- Embedded file name
- Encryption algorithm
- Whether data exists

STEP 5: Extract the Hidden Message

steghide extract -sf stego.jpg

- Enter the same passphrase
- File will be extracted (e.g., `secret.txt`)

STEP 6: Verify Message Integrity

Method 1: Using `diff`

```
diff secret.txt secret.txt
```

- No output → files are identical ✅

Method 2: Using `md5sum`

```
md5sum secret.txt
```

Example Output:

5d41402abc4b2a76b9719d911017c592 secret.txt

✅ Same hash → message verified successfully

STEP 7: Verify Image Integrity (LSB Check)

```
md5sum cover.jpg
md5sum stego.jpg
```

Example Output:

5d41402abc4b2a76b9719d911017c592 cover.jpg
7a8b9c1d2e3f456789abcd1234567890 stego.jpg

Interpretation:

- Hash values are different → file content changed
- Change is due to LSB embedding
- Image looks visually unchanged 👁️

Conclusion

The experiment successfully demonstrated **image steganography using Steghide** based on the **Least Significant Bit (LSB)** technique, ensuring hidden data transmission without noticeable visual changes.

Ex.No: 8.2

Hashing & HMAC

Aim:

To compute the hash of a file using SHA-256, generate an HMAC using a secret key, and verify file integrity using hashing techniques.

Basic Concepts

Hashing

A cryptographic hash function converts input data of any size into a fixed-length hash value (digest).

Flow:

```
Input File → Hash Function → 256-bit Hash Value
```

Properties of Hashing:

- Fixed output length
- One-way function (cannot reverse)
- Small change in input → completely different hash
- Used for integrity verification

Example:

File content: Hello SHA-256
Hash: 185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969

HMAC (Hash-based Message Authentication Code)

HMAC combines:

- Secret key
- Message
- Hash function

Provides:

- Integrity
- Authentication

Formula:
HMAC = Hash(Secret Key + Message)

Procedure

Step 1: Create a Sample File

```
echo "This is a sample file for hashing experiment" > file.txt
```

Check content:

```
cat file.txt
```

Step 2: Compute SHA-256 Hash

```
sha256sum file.txt
```

Example Output:

c0535e4be2b79ffd93291305436bf889314e4a3faec05ecffcbb9c3e3c9b9f8c file.txt

Save hash:

```
sha256sum file.txt > hash.txt
```

Step 3: Generate HMAC

```
openssl dgst -sha256 -hmac "secretkey" file.txt
```

Example Output:

HMAC-SHA256(file.txt)= 8f6b1c5d8a5b7a1e9c5f7b2a4e6d3f9c1e7a6b5c4d3e2f1a0b9c8d7e6f5a4b3

Explanation:

- `-sha256` → hashing algorithm
- `-hmac` → secret key

Step 4: Verify File Integrity

```
sha256sum -c hash.txt
```

Output:

file.txt: OK

✅ File is not modified

Step 5: Modify the File

```
echo "File modified!" >> file.txt
```

Step 6: Verify Again

```
sha256sum -c hash.txt
```

Output:

file.txt: FAILED
sha256sum: WARNING: 1 computed checksum did NOT match

❌ File has been modified

HMAC Authentication

HMAC = Hash(Key + Message)

Only someone with the secret key can generate the correct HMAC.

Ensures:

- Message integrity
- Sender authentication

Conclusion

The SHA-256 hash and HMAC of the file were successfully generated and verified.
File integrity checking using hashing techniques was demonstrated effectively.

Ex No: 9.2

SQL Injection – Sample Program

Aim:

To demonstrate SQL Injection vulnerability and its prevention using parameterized queries.

🔷 Step 1: Create Database (in a cell)

```
import sqlite3

conn = sqlite3.connect("test.db")
cur = conn.cursor()

cur.execute("CREATE TABLE users (id INTEGER, username TEXT)")
cur.execute("INSERT INTO users VALUES (1, 'alice')")
cur.execute("INSERT INTO users VALUES (2, 'bob')")
```

```
conn.commit()
conn.close()
```

🔷 Step 2: Vulnerable Code

```
import sqlite3

name = input("Enter username: ")

conn = sqlite3.connect("test.db")
cur = conn.cursor()

query = "SELECT * FROM users WHERE username = '" + name + "';"
print("Executing:", query)

cur.execute(query)
print(cur.fetchall())

conn.close()
```

🔷 Step 3: Test Normal Input

```
alice
```

👉 Output:

```
[(1, 'alice')]
```

🔷 Step 4: SQL Injection Attack

```
' OR '1'='1
```

👉 Output:

```
[(1, 'alice'), (2, 'bob')]
```

💀 Boom — vulnerability shown.

🔷 Step 5: Secure Version

```
import sqlite3

name = input("Enter username: ")

conn = sqlite3.connect("test.db")
cur = conn.cursor()

cur.execute("SELECT * FROM users WHERE username = ?", (name,))
print(cur.fetchall())

conn.close()
```

🔷 Step 6: Test Attack Again

```
' OR '1'='1
```

👉 Output:

```
[]
```

✅ Attack blocked.

Result:

SQL Injection vulnerability was demonstrated and prevented using parameterized queries.